Classes

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
1 Hr		

1. Animals make sounds, have a weight (kg) and sizes (cm).
   - Cats, fish and rabbits make different noices when rubbed: 'miouw', 'blub' and 'ribbit'. Other animals remain silent.
   - We can ask an animal if it is too heavy.
     - a cat is too heavy if it's weight is bigger than it's size.
     - fish are never too heavy
     - rabbits are too heavy if bigger than one kilogram
     - in general animals are too heavy when their weight is bigger than their size - 10;
   - An animal must be able to give (using toString()) it's full type and wether it is too heavy:
     "Hey, I'm a fish and I'm not Fat !"
     "Hey, I'm a cat and I need to go and see the Weightwatchers"
   - Two animals are considered equal if they Body Mass Index is equal
     BMI: weight / (size * size).

   Create your testClass(es).

   TIPS :
   - use constants where possible.
   - use super. in toString() of the different subclasses.

   BRAINTEASER:
   - How could you implement the toString() in the super class (Animal) and still have the correct toString() without the need to override the toString in the subclasses.

Classes

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
2 Hr		

2. We are going to model a ship for transporting cars over the canal.
   - This ship has a maximum capacity of 100 places.
   - A second limitation is that the maximum weight that can be supported is 95000 kg.
   - We are going to model two kind of vehicles.
     - A car has a weight of 1000 kg and takes one place.
     - A van has a weight of 1500 kg and takes two places.
   - Create a class `Ship`.
   - Create the variables necessary to hold the information of the ship.
   - Write a method `addVehicle(Vehicule aVehicule)` to add a vehicule into the ship.
   - Write a method `totalWeight()` that returns the total weight of the content of the ship.
   - Write a method `removeVehicle()` to remove the last added vehicle
   - Write a method getContent() to check the content of the ship (return a String).
     - for example : "2500 kg and 25 places"
   - When adding or removing cars to your ship, pay attention to the maximum total number of cars and the maximum total weight.

Classes

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
45’		

3. The results of a survey of the households in your township have been made available.
   - Define a class `Household` that contains data for one household, including a four-digit integer identification number, the annual income for the household, and the number of members of the household.
   - Define a class `Township` that stores a number of households in a collection.
   - Write some methods in this class to perform the following analyses:
     - Count the number of households included in the township and print a table displaying the data read in.
     - Calculate the average household income, and list the identification number and income of each household that exceeds the average.
     - Determine the percentage of households having incomes below the poverty level.
       - The poverty level can be computed using the formula *p = $6500.00 + $750.00 (m - 2)* where **m** is the number of members of each household.
       - This formula shows that the poverty level depends on the number of family members, m, and that the poverty level increases as m gets larger.
   - Test your program on the following data.

Identification number	Annual income	Household members
1041	$12,180	4
1062	$13,240	3
1327	$19,800	2
1483	$22,458	8
1900	$17,000	2
2112	$18,125	7
2345	$15,623	2
3210	$3,200	6
3600	$6,500	5
3601	$11,970	2
4725	$8,900	3
6217	$10,000	2
9280	$6,200	1

Create the appropriate testClass(es).

Classes

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
2Hr		

4. We want you to write an application for the HR division of a departmement of the defense.
   - Write a class to represent a member of the defense (DefensePers).
     - For each defense pers, we must know:
       - › the matriculation.
       - › the name.
       - › the birth year.
     - Write methods able to:
       - › calculate the age of each person.
       - › return a representation of the person as a string like this : "name (matriculation)".
       - › determine if a person must go to the pension (by default, it is when the age is more than 65).
   - Write a sub class of DefensePers: CivilianPers
     - This class must know:
       - › The sex of the person (i.e. if the pers is a femal).
     - Write methods able to:
       - › return a representation of the person as a string like this : "Mr name (matriculation)" if a man and "Mm name (matriculation)" if a femal .
       - › determine if a person must go to the pension (actually, the limit is 60 for the mans and 50 for the femals).
   - Write a sub class of DefensePers: Military.
     - This class must know:
       - › The rank of the military. For a military, the pension limit depends on the rank.
         So, you must write a class Rank. for each rank , we must know: a name and the pension limit
     - Write methods able to:
       - › return a representation of the person as a string like this : "RANK name (matriculation)"
       - › determine if a person must go to the pension (For a military, the pension limit depends on the rank !!).
   - Write a class to represent a military unit.
     - Each unit has a name
     - Each unit can contain personel of the defense (militaries and civilians)
     - Write methods to:
       - › add a defense pers in the unit
       - › remove a defense pers from the unit
       - › return a list containing all the defense pers that must go to pension.

   HINTS:
   - Use a static field to represent the actual year.
   - The representation method could be the toString() method of the class Object.

Classes

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
1 Hr 20’		

5. We are going to model a thriatlete.
   - A triathlete is someone who swims, runs and rides a bike.
     - We want to know about a triathlete everything that we know about swimmers, runners and cyclists.
   - A cyclist is a person who rides on a bike.
     - Certain cyclist love to ride in the mountains and are good climbers.
     - Certain are very good in riding against time.
     - To control if a cyclist is in good shape, the doctors often take samples of the “redBodyRate” (hematocriteRate) of the cyclist. If this rate is above 50, the cyclist gets an interdiction to participate in games.
     - A cyclist must keep track of his maximum heart rate.
   - A swimmer swims in the water of course.
     - Certain people like to swim crawl, other people like to swim breast, some people like both.
     - Each swimmer has a certain preferred distance as specialty.
     - To get better results, the swimmer also wants to keep track of his maximum heart rate.
   - A runner is known for running on land.
     - He is characterized by the length of his step.
     - When running, for ideal increase of his condition, he keeps track of his heart rate.
     - Each runner has a certain preferred distance as specialty.
   - Implement the triathlete so that we know all of his information. Choose the right kind of classes for your hierarchy.

   Create the appropriate testClass(es).

Classes

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
1 Hr 30’		

6. An estate office spends its time selling houses. To automate some of their jobs, they want to model their houses.
   - There are two things that they are interested in when storing their houses.
     - They want to know the number of rooms.
     - They want to be able to communicate the total surface of a house.
     - They want to be able to communicate the price.
   - A particular type of house is a villa.
     - To be able to calculate the two properties of a house mentionned above, we are going to model a villa as being a collection of rooms.
     - Each of these rooms knows its surface.
     - Knowing the number of rooms in a villa, and the surface of each of these rooms, we are now able to calculate the two demanded answers.
     - To calculate the price:
       › surface * price per unit (+10 % if there is a big garden).
     - The estate office also wants to know if the villa has a big garden.
     - Hint: use a `Collection` type to store the different rooms of the villa.
   - Another kind of house is a loft.
     - A loft naturally has no garden, but a loft can have a balcony.
     - The estate office wants to be able to communicate the size of the balcony.
     - When asking for the number of rooms, there will always be just one room.
     - That makes it very easy to tell us the surface of the loft.
     - To calculate the price:
       › surface * price per unit (+5 % if there is a balcony).
   - When modelling other types of houses, we want to be sure that the two questions from above (number of rooms and total surface) will be provided for that type of house.
     - How can this be achieved?
   - Implement all this information in a suitable hierarchy, using the appropriate kind of class for each object type.

   Create the appropriate testClass(es).

Elementary Control Structures

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
50’		

7. Add a method asEnglish to class `MyNumber` that returns a string representing the English form of the number.
   - To make the task simpler, you may wish to restrict the integer receiving the message to values in the range 0 to 999.
   - Remarks:
     - Don’t forget to treat the number 0.
     - Pay attention to the word “and”. Here are some examples that reflect when to put “and” and when not.

       ```
       twelve, fiftynine, two hundred and twentysix, three hundred
       ```

   - The following code should help you getting started

```
public class MyNumber{
   private static String[] digits =
         {"nil", "one", "two", "three", "four", "five", "six",
          "seven", "eight", "nine", "ten", "eleven", "twelve",
          "thirteen", "fourteen", "fifteen", "sixteen", "seventeen",
          "eighteen", "ninteen", "twenty"};
   private static String[] tens =
         {"twenty", "thirty", "fourty", "fifty", "sixty",
          "seventy", "eighty", "ninety"};

   private int number;

   public MyNumber(int number){
      this.number = number;
   }

   public String asEnglish(){

         //Your code here//
   }
}
```

   - Don’t be afraid to add more methods to break your code into little pieces. These methods can be called by the `asEnglish()` method to do the work that has to be done (divida et impera).

   Create the appropriate testClass.

Classes

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
40’		

8. Create a class `Car` that will model an existing car and its gear box.
   - This class will have an instance variable speed to keep track of the speed of your car.
   - Beside this variable we also need an instance variable to know our current gear.
   - We need a method `accelerate()` that will make the car go faster.
     - The rate at which we increase our speed will depend on the current gear.
     - The following table gives us the different values of increasing our speed.

gear	increase (mph)
0(neutral)	0
1	5
2	10
3	15
4	20
5	30

     - Each time you send the message `accelerate()` to a car will increase its current speed by the value mentionned above.
   - Limit speed for gear are as follow:

gear	min	max
0(neutral)	0	0
1	0	20
2	20	40
3	40	60
4	60	100
5	100	-

     - if you are below or over a limit after an acceleration or a deceleration, you first adapt the speed with the in(de)creasing speed of the current gear and then adapt the gear.
     - when you have just a limit speed, you can choose your gear.
   - In the same way, we can implement a method `slowDown()` that will subtract the value of the table from our current speed.
   - You have to implement a method `start()` to start the car. You can then call the `gearUp()` method to put the first gear.

- The user can only use `gearUp()` when the speed is 0 and the gear is 0 (otherwise thrown a checked exception `InvalidCallOfGearUpException`)
- The user can only use `gearDown()` when the speed is 0 and the gear is 1 (otherwise thrown a checked exception `InvalidCallOfGearDownException`)
- When your gear is in neutral position and you try to accelerate, nothing happens of course.
- Comment your actions. When sending one of these methods, put some information on the screen, such as:

```
Current gear : 1
Current speed : 0
Accelerating. Current speed: 5.
Changing gear. Current gear: 2.
Accelerating. Current speed: 15 mph.
Accelerating. Current speed: 25 mph.
Changing gear. Current gear: 3.
Accelerating.  Current speed: 40 mph.
...
```

Create the appropriate testClass(es).

Classes

Engineering task		
PairMembers:		
Expected Time by Instructor	Estimated Time by Student	Real Time worked on task
60’		

9. The determinant of a matrix:

$$|A| = \sum_{i=1}^{k} a_{ij} C_{ij}, \qquad C_{ij} \equiv (-1)^{i+j} M_{ij}.$$

- Make a class Matrix with
  - a constructor that takes in a number that gives the size of the matrix (all matrixes are square).
  - a method to introduce the different elements of the matrix.
  - a toString() method that gives a nice representation of the matrix.
  - a method that gives the determinant of the matrix
  - a method that indicates if the matrix is singular (it’s determinant is 0)
- For our problem domain, matrixes may be considered equal if there determinants are multiples of each other.

Write the testClasse(s) !

BRAINTEASER:

- Optimalize the initialization of the different matrixes. You will need to determine all the minors of a determinant. Find a way to avoid the re-initialization of the values of the matrix.